

# Entregável de Desenvolvimento

Davi Rosendo Carvalho

## 1. O que foi feito

Eu desenvolvi as funcionalidades de gerenciamento da API da Folhagem & Cia. Minha primeira ação foi criar um template a partir do repositório base que foi enviado pelos desenvolvedores do CITi, e percebi que o projeto utiliza Node.js, Express, TypeScript, Prisma, Postgres como banco e Docker para conteinerização.

A partir disso, foram criadas as sete rotas:

POST /plantas → cadastro de plantas

GET /plantas → listagem com filtros opcionais

GET /plantas/:id → consulta de uma planta

PATCH /plantas/:id → atualização parcial

DELETE /plantas/:id → exclusão

POST /plantas/:id/venda → venda com redução do estoque

GET /plantas/estoque/valor-total → cálculo do valor do estoque e total de vasos

Além de disponibilizar essas rotas, eu implementei as validações necessárias para que a API recebesse dados compatíveis com as regras do desafio. No cadastro, todos os campos da planta são obrigatórios, enquanto na edição somente os campos enviados são validados e alterados. Foram incluídas verificações para textos vazios, valores numéricos, preço maior que zero, estoque não negativo, intervalo de rega de pelo menos um dia e luminosidade limitada aos valores do enum. Também foram tratados os casos de IDs inválidos, plantas inexistentes e filtros incorretos, buscando retornar mensagens claras e os status HTTP previstos no enunciado.

A venda inclui uma atualização condicionada à disponibilidade no banco, para impedir que duas compras simultâneas deixem o estoque negativo.

Como complemento, foi criada uma suíte de testes HTTP automatizados e uma página de evidências https://davircarvalh0.github.io/repositorio-desafio-dev/, com requisições, respostas e resultados. Também foi configurado um fluxo do GitHub Actions para executar as verificações e publicar a página no GitHub Pages.

## 2. Como foi feito

Eu segui os passos que o desafio tinha proposto, ou seja, a criação do model e da migration, implementação do Repository, construção do Controller, registro das rotas, inclusão dos filtros, implementação da venda e cálculo do valor total do estoque. O CRUD de usuários disponibilizado no projeto serviu como referência para os nomes das funções, a estrutura dos arquivos e a comunicação entre as camadas.

A primeira alteração foi feita no schema do Prisma, com a criação do model Planta e do enum Luminosidade. Os campos foram definidos de acordo com o desafio, incluindo o identificador gerado automaticamente, as informações de cuidado da planta e as datas de criação e atualização. O preço foi representado como um número inteiro em centavos, evitando o uso de valores decimais nas operações com dinheiro. Depois da definição do model, a migration foi gerada e incluída no histórico do projeto para que a estrutura do banco pudesse ser reproduzida em outros ambientes.

Na organização do código, mantive a separação entre rotas, Controller e Repository. As rotas conectam os endereços HTTP às funções responsáveis por atendê-los, o Controller recebe os dados, realiza as validações e define o status e o conteúdo da resposta e o Repository concentra as consultas e alterações feitas por meio do Prisma. Essa divisão ajuda a localizar problemas e evita espalhar o acesso ao banco pela aplicação. Também ajuda a compreender o caminho de uma requisição, desde sua chegada à API até a leitura ou alteração dos registros no PostgreSQL.

Uma decisão importante nas validações foi diferenciar a ausência de um campo de um valor que pode ser considerado falso em JavaScript. Em vez de utilizar apenas uma condição, a presença dos dados foi verificada por comparação com undefined. Isso permite aceitar corretamente uma planta com estoque zero ou com petFriendly: false. A validação também considera o tipo recebido, de modo que um preço enviado como texto não seja aceito como número. No PATCH, os campos não enviados permanecem sem alteração, enquanto os campos presentes passam pelas mesmas regras de conteúdo utilizadas no cadastro.

Os filtros da listagem também exigiram uma conversão cuidadosa, pois os parâmetros da URL chegam como texto. O valor de petFriendly é validado para aceitar somente "true" ou "false" e depois convertido para booleano. Quando o parâmetro não é informado, ele permanece ausente da condição da consulta, evitando que uma listagem sem filtros seja interpretada como uma busca apenas por plantas não seguras para pets.

## 3. Principais dificuldades

Apesar de eu já ter passado em cadeiras como Desenvolvimento de Software e estar fazendo IESI, que são cadeiras que utilizam grande parte da stack desse desafio, eu ainda tenho um pouco de dificuldade de conseguir configurar todos os ambientes, principalmente a conteinerização com Docker. Inicialmente, eu tive um problema com a porta, pois ela já estava sendo usada em outro projeto com o Docker, então eu tive que modificar o .env para que o desafio tivesse uma porta própria.

Além disso, um ponto da implementação que exigiu atenção foi compreender a diferença entre verificar se um campo foi enviado e verificar se seu valor é válido. Ao analisar, precisei entender por que uma condição como if (!campo) não seria adequada para todos os dados da planta. Em JavaScript, tanto 0 quanto false são considerados falsos nessa comparação, mas são valores permitidos pelo desafio: uma planta pode estar com estoque zerado e pode não ser segura para animais. Com o apoio da inteligência artificial (ChatGPT), vi a necessidade de comparar o campo com undefined para identificar sua ausência e, em seguida, verificar seu tipo e conteúdo separadamente.

## 4. Bônus feitos

Após concluir os requisitos obrigatórios, implementei os quatro bônus do desafio. A busca por nome permite encontrar plantas sem diferenciar letras maiúsculas e minúsculas, enquanto a ordenação organiza os resultados de forma crescente por preço ou nome. Esses recursos podem ser combinados com os filtros de luminosidade e petFriendly. Também adicionei a rota GET /plantas/esgotadas, que retorna somente plantas com estoque igual a zero.

O script de seed, executado com yarn seed, insere exemplos de Costela-de-adão, Samambaia e Cacto. Ao ser executado novamente, verifica os registros existentes para evitar duplicações e preservar alterações de preço ou estoque. Com o apoio do Codex, foram acrescentados testes para os quatro bônus, incluindo parâmetros inválidos e repetição do seed. A suíte passou de 70 para 94 verificações,

todas aprovadas na execução local registrada, com os resultados apresentados na página de evidências.

## 5. Uso de inteligência artificial

No uso de inteligência artificial, eu utilizei o ChatGPT para que ele fosse me auxiliando em cada passo do projeto, garantindo que eu obtivesse o melhor resultado. Além disso, fiz questão que ele me explicasse cada parte que eu não tinha entendido para melhor aproveitamento do projeto. Com o Codex, eu criei a página no GitHub Pages e criei os testes automáticos a cada commit, e, por fim, pedi ajuda para a IA revisar tanto minha branch no repositório quanto meu relatório, para garantir que tudo esteja de acordo com o que foi pedido no projeto.